

HW5 - GAN

Problem 1

Build and train a Generative Adversarial Network (GAN) to generate handwritten digits from MNIST dataset.

Data: The MNIST dataset contains 70,000 images of handwritten digits between 0 and 9 with 28×28 pixel size. Most deep learning libraries, such as Keras, provide access to the MNIST dataset.

```
In [1]: # Import Libraries
import shutil
from google.colab import files
import os

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import random_split, DataLoader, TensorDataset
from IPython.display import display, Image
import time
import copy
from datetime import datetime
import pytz
```

```
In [2]: # Import dataset (MNIST)

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f"[{timestamp}] Data Load Start")

from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```

BUFFER_SIZE = len(train_dataset)
BATCH_SIZE = 64

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f"\n[{timestamp}] Data Ready (MNIST train set loaded)")

```

[23:21:21] Data Load Start

```

100%|██████████| 9.91M/9.91M [00:02<00:00, 4.80MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 129kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.22MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 15.0MB/s]

```

[23:21:34] Data Ready (MNIST train set loaded)

Architecture: Please build a GAN by combining the discriminator and generator models, such that the discriminator follows the generator in a sequential setting.

The first step is to define the discriminator model. The discriminator solves the binary classification problem by determining whether the input image is real or fake. Please build a discriminator by adding the layers to the Sequential model using the configuration below.

1. Conv2D

- Filters: 32
- Kernel size: (3,3)
- Activation: ReLU

2. MaxPooling2D

- Pool size: (2,2)

3. Conv2D

- Filters: 64
- Kernel size: (3,3)
- Activation: ReLU

4. MaxPooling2D

- Pool size: (2,2)

5. Flatten

6. Dense

- Units: 1
- Activation: Sigmoid

In [3]: *# Discriminator Model Definition*

```

class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()

```

```

self.model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),

    nn.Conv2d(32, 64, kernel_size=3),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),

    nn.Flatten(),
    nn.Linear(64 * 5 * 5, 1),
    nn.Sigmoid()
)

def forward(self, x):
    x = self.model(x)
    return x

```

Second step involves building the generator model. The generator model is responsible for creating new fake but plausible images of handwritten digits. Build a generator by adding layers to the Sequential model using the configuration below.

1. Dense

- Units: 128

2. LeakyReLU

- Alpha: 0.2

3. Reshape

- Size: (7,7,128)

4. Conv2DTranspose

- Filters: 128
- Kernel size: (4,4)
- Strides: (2,2)
- Padding: Zero padding

5. LeakyReLU

- Alpha: 0.2

6. Conv2DTranspose

- Filters: 1
- Kernel size: (7,7)
- Strides: (2,2)
- Activation: tanh

- Padding: Zero padding

In [4]: *# Generator Model Definition*

```
class Generator(nn.Module):
    def __init__(self, noise_dim=100):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(noise_dim, 128 * 7 * 7),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Unflatten(1, (128, 7, 7)),

            nn.ConvTranspose2d(128, 128, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.ConvTranspose2d(128, 1, kernel_size=7, stride=2, padding=3, output_p
            nn.Tanh()
        )

    def forward(self, x):
        return self.model(x)
```

In [5]: *# Models testing*

```
timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f"[{timestamp}] Models Test Start")

generator = Generator()

noise = torch.from_numpy(np.random.normal(size=(1, 100)).astype(np.float32))
generator.eval()
with torch.no_grad():
    generated_image = generator(noise)

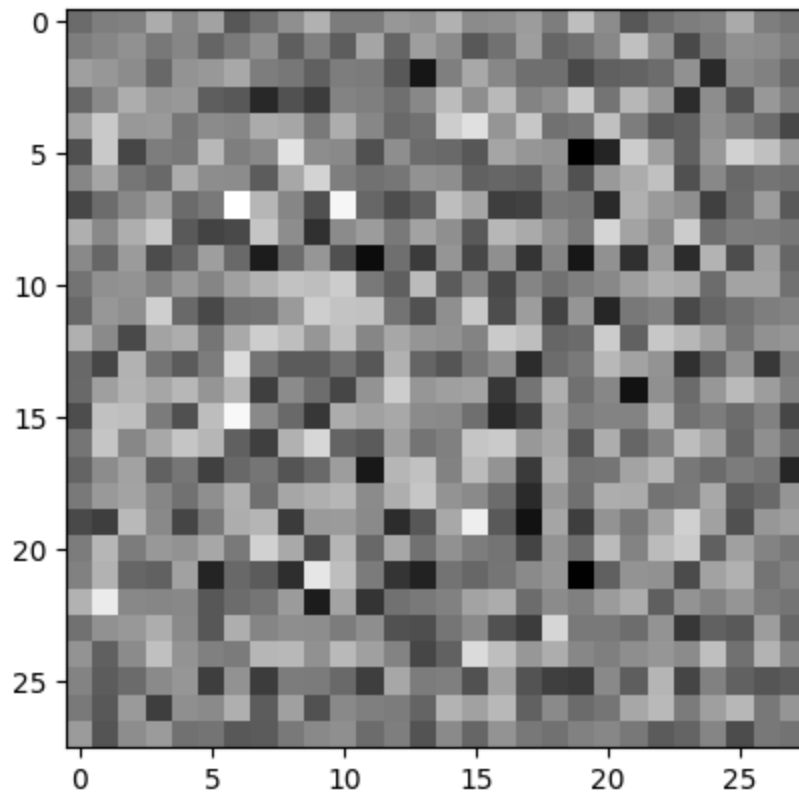
image_to_plot = generated_image[0, 0, :, :].cpu().numpy()

plt.imshow(image_to_plot, cmap='gray')
plt.show()

discriminator = Discriminator()
decision = discriminator(generated_image)
print (decision)

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f"[{timestamp}] Models Test Finish")
```

[23:21:34] Models Test Start



```
tensor([[0.5294]], grad_fn=<SigmoidBackward0>)
[23:21:35] Models Test Finish
```

Training: The GAN should be compiled with an ADAM optimizer to minimize the binary cross entropy. Fake samples are generated by the generator by using random noise as input, while the real images are randomly selected from the MNIST dataset. The combined real and fake images, along with their respective labels, are passed to the discriminator model for training.

In [6]: *# Set up device and models*

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

generator = Generator().to(device)
discriminator = Discriminator().to(device)

criterion = nn.BCELoss()
gen_optimizer = optim.Adam(generator.parameters(), lr=1e-4)
disc_optimizer = optim.Adam(discriminator.parameters(), lr=1e-4)
```

In [7]: *# Single epoch train step*

```
def train_step(real_images, noise_dim=100):
    batch_size = real_images.size(0)

    real_labels = torch.ones(batch_size, 1, device=device)
    fake_labels = torch.zeros(batch_size, 1, device=device)

    gen_optimizer.zero_grad()
    noise = torch.randn(batch_size, noise_dim, device=device)
    fake_images = generator(noise)
```

```

fake_outputs = discriminator(fake_images)
gen_loss = criterion(fake_outputs, real_labels)
gen_loss.backward()
gen_optimizer.step()

disc_optimizer.zero_grad()
real_outputs = discriminator(real_images)
real_loss = criterion(real_outputs, real_labels)
fake_outputs = discriminator(fake_images.detach())
fake_loss = criterion(fake_outputs, fake_labels)
disc_loss = real_loss + fake_loss
disc_loss.backward()
disc_optimizer.step()

return gen_loss.item(), disc_loss.item()

```

In [8]: *# Loops*

```

import matplotlib.pyplot as plt
from torchvision.utils import save_image

def generate_and_save_images(model, epoch, test_input):
    os.makedirs('./runs', exist_ok=True)
    model.eval()
    with torch.no_grad():
        generated = model(test_input).cpu()
    save_image(generated, f'./runs/image_at_epoch_{epoch:04d}.png', normalize=True)
    model.train()

# PLOT LOSS CURVES
def plot_loss(curve, loss_name, output_dir="Plot JPGs"):
    os.makedirs(output_dir, exist_ok=True)

    plt.figure(figsize=(8, 5))
    epochs = np.arange(1, len(curve) + 1)
    plt.plot(epochs, curve, marker='o', linewidth=1.5, label=loss_name)
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"{loss_name} Curve")
    plt.grid(True, linestyle='--', alpha=0.4)
    plt.legend()
    plt.tight_layout()

    filename = os.path.join(output_dir, f"{loss_name.replace(' ', '_')}.jpg")
    plt.savefig(filename, dpi=150)
    plt.close()

def train(generator, discriminator, train_loader, epochs, noise_dim=100, save_every
os.makedirs(checkpoint_dir, exist_ok=True)
fixed_noise = torch.randn(16, noise_dim, device=device)

gen_loss_history = []
disc_loss_history = []

for epoch in range(1, epochs + 1):
    start_time = time.time()

```

```

gen_loss_total = 0.0
disc_loss_total = 0.0

for real_images, _ in train_loader:
    real_images = real_images.to(device)
    gen_loss, disc_loss = train_step(real_images, noise_dim)
    gen_loss_total += gen_loss
    disc_loss_total += disc_loss

avg_gen_loss = gen_loss_total / len(train_loader)
avg_disc_loss = disc_loss_total / len(train_loader)

gen_loss_history.append(avg_gen_loss)
disc_loss_history.append(avg_disc_loss)

generate_and_save_images(generator, epoch, fixed_noise)

if epoch % save_every == 0:
    torch.save({
        'epoch': epoch,
        'generator_state_dict': generator.state_dict(),
        'discriminator_state_dict': discriminator.state_dict(),
        'gen_optimizer_state_dict': gen_optimizer.state_dict(),
        'disc_optimizer_state_dict': disc_optimizer.state_dict(),
    }, os.path.join(checkpoint_dir, f'checkpoint_epoch_{epoch}.pth'))

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f'[{timestamp}] Epoch [{epoch}/{epochs}] Gen Loss: {avg_gen_loss:.4f}

# Save one dedicated snapshot from the final trained generator.
generator.eval()
with torch.no_grad():
    final_generated = generator(fixed_noise).cpu()
save_image(final_generated, './runs/final_epoch_samples.png', normalize=True)
generator.train()

print("Training complete")

plot_loss(gen_loss_history, "Generator Loss")
plot_loss(disc_loss_history, "Discriminator Loss")

print("Plots saved")
print("Saved final generator snapshot: ./runs/final_epoch_samples.png")

return gen_loss_history, disc_loss_history

```

In [9]: *## EXPERIMENT*

```

MAX_EPOCHS = 100

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f'[{timestamp}] Run Start")

gen_loss_history, disc_loss_history = train(generator, discriminator, dataloader, e

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")

```

```
print(f"[{timestamp}] Run Finish")

# Zip and download loss plots immediately after training
plot_zip_dir = "Plot JPGs"
if os.path.isdir(plot_zip_dir):
    shutil.make_archive("Plot_JPGs", "zip", plot_zip_dir)
    print("Created Plot_JPGs.zip")
    if files is not None:
        try:
            files.download("Plot_JPGs.zip")
        except Exception as e:
            print(f"Download skipped or failed: {e}")
    else:
        print(f"Plot folder not found: {plot_zip_dir}")

timestamp = datetime.now(pytz.timezone('US/Eastern')).strftime("%H:%M:%S")
print(f"[{timestamp}] Loss plots exported")
```


[23:21:35]	Run Start				
[23:21:44]	Epoch [1/100]	Gen Loss: 5.2445	Disc Loss: 0.1838	Time: 9.14s	
[23:21:52]	Epoch [2/100]	Gen Loss: 8.4792	Disc Loss: 0.0007	Time: 8.00s	
[23:22:00]	Epoch [3/100]	Gen Loss: 4.7785	Disc Loss: 0.0855	Time: 8.04s	
[23:22:08]	Epoch [4/100]	Gen Loss: 5.5431	Disc Loss: 0.0138	Time: 8.00s	
[23:22:16]	Epoch [5/100]	Gen Loss: 4.8723	Disc Loss: 0.0533	Time: 8.11s	
[23:22:24]	Epoch [6/100]	Gen Loss: 2.8527	Disc Loss: 0.3359	Time: 8.08s	
[23:22:32]	Epoch [7/100]	Gen Loss: 1.8715	Disc Loss: 0.6082	Time: 8.00s	
[23:22:40]	Epoch [8/100]	Gen Loss: 1.8288	Disc Loss: 0.6108	Time: 7.90s	
[23:22:48]	Epoch [9/100]	Gen Loss: 1.8006	Disc Loss: 0.5868	Time: 7.93s	
[23:22:56]	Epoch [10/100]	Gen Loss: 1.8374	Disc Loss: 0.5891	Time: 7.94s	
[23:23:04]	Epoch [11/100]	Gen Loss: 1.7422	Disc Loss: 0.6311	Time: 8.17s	
[23:23:13]	Epoch [12/100]	Gen Loss: 1.6563	Disc Loss: 0.7293	Time: 8.15s	
[23:23:21]	Epoch [13/100]	Gen Loss: 1.4795	Disc Loss: 0.8105	Time: 8.25s	
[23:23:29]	Epoch [14/100]	Gen Loss: 1.3788	Disc Loss: 0.8510	Time: 8.05s	
[23:23:37]	Epoch [15/100]	Gen Loss: 1.3308	Disc Loss: 0.8736	Time: 7.93s	
[23:23:45]	Epoch [16/100]	Gen Loss: 1.3252	Disc Loss: 0.8675	Time: 8.11s	
[23:23:53]	Epoch [17/100]	Gen Loss: 1.3455	Disc Loss: 0.8507	Time: 8.07s	
[23:24:01]	Epoch [18/100]	Gen Loss: 1.3700	Disc Loss: 0.8304	Time: 8.03s	
[23:24:09]	Epoch [19/100]	Gen Loss: 1.3968	Disc Loss: 0.8093	Time: 8.02s	
[23:24:17]	Epoch [20/100]	Gen Loss: 1.4244	Disc Loss: 0.7930	Time: 8.09s	
[23:24:25]	Epoch [21/100]	Gen Loss: 1.4579	Disc Loss: 0.7794	Time: 8.06s	
[23:24:33]	Epoch [22/100]	Gen Loss: 1.4849	Disc Loss: 0.7679	Time: 8.10s	
[23:24:41]	Epoch [23/100]	Gen Loss: 1.5059	Disc Loss: 0.7614	Time: 7.91s	
[23:24:49]	Epoch [24/100]	Gen Loss: 1.5229	Disc Loss: 0.7582	Time: 7.93s	
[23:24:57]	Epoch [25/100]	Gen Loss: 1.5428	Disc Loss: 0.7541	Time: 8.13s	
[23:25:05]	Epoch [26/100]	Gen Loss: 1.5519	Disc Loss: 0.7564	Time: 7.85s	
[23:25:13]	Epoch [27/100]	Gen Loss: 1.5613	Disc Loss: 0.7554	Time: 8.07s	
[23:25:21]	Epoch [28/100]	Gen Loss: 1.5707	Disc Loss: 0.7578	Time: 8.03s	
[23:25:29]	Epoch [29/100]	Gen Loss: 1.5763	Disc Loss: 0.7609	Time: 8.05s	
[23:25:37]	Epoch [30/100]	Gen Loss: 1.5800	Disc Loss: 0.7666	Time: 7.83s	
[23:25:45]	Epoch [31/100]	Gen Loss: 1.5855	Disc Loss: 0.7665	Time: 7.84s	
[23:25:53]	Epoch [32/100]	Gen Loss: 1.5758	Disc Loss: 0.7727	Time: 8.00s	
[23:26:01]	Epoch [33/100]	Gen Loss: 1.5796	Disc Loss: 0.7768	Time: 8.00s	
[23:26:09]	Epoch [34/100]	Gen Loss: 1.5751	Disc Loss: 0.7795	Time: 7.93s	
[23:26:17]	Epoch [35/100]	Gen Loss: 1.5771	Disc Loss: 0.7824	Time: 8.25s	
[23:26:25]	Epoch [36/100]	Gen Loss: 1.5676	Disc Loss: 0.7918	Time: 8.29s	
[23:26:33]	Epoch [37/100]	Gen Loss: 1.5669	Disc Loss: 0.7990	Time: 8.01s	
[23:26:41]	Epoch [38/100]	Gen Loss: 1.5597	Disc Loss: 0.8002	Time: 7.89s	
[23:26:49]	Epoch [39/100]	Gen Loss: 1.5569	Disc Loss: 0.8062	Time: 8.03s	
[23:26:57]	Epoch [40/100]	Gen Loss: 1.5487	Disc Loss: 0.8103	Time: 8.02s	
[23:27:05]	Epoch [41/100]	Gen Loss: 1.5421	Disc Loss: 0.8150	Time: 7.94s	
[23:27:13]	Epoch [42/100]	Gen Loss: 1.5456	Disc Loss: 0.8160	Time: 7.86s	
[23:27:21]	Epoch [43/100]	Gen Loss: 1.5431	Disc Loss: 0.8236	Time: 8.29s	
[23:27:30]	Epoch [44/100]	Gen Loss: 1.5362	Disc Loss: 0.8271	Time: 8.27s	
[23:27:38]	Epoch [45/100]	Gen Loss: 1.5328	Disc Loss: 0.8259	Time: 8.22s	
[23:27:46]	Epoch [46/100]	Gen Loss: 1.5255	Disc Loss: 0.8331	Time: 8.19s	
[23:27:54]	Epoch [47/100]	Gen Loss: 1.5303	Disc Loss: 0.8349	Time: 8.23s	
[23:28:03]	Epoch [48/100]	Gen Loss: 1.5198	Disc Loss: 0.8378	Time: 8.27s	
[23:28:11]	Epoch [49/100]	Gen Loss: 1.5167	Disc Loss: 0.8358	Time: 8.26s	
[23:28:19]	Epoch [50/100]	Gen Loss: 1.5150	Disc Loss: 0.8416	Time: 8.23s	
[23:28:27]	Epoch [51/100]	Gen Loss: 1.5143	Disc Loss: 0.8377	Time: 8.20s	
[23:28:35]	Epoch [52/100]	Gen Loss: 1.5050	Disc Loss: 0.8456	Time: 8.15s	
[23:28:44]	Epoch [53/100]	Gen Loss: 1.5039	Disc Loss: 0.8457	Time: 8.11s	
[23:28:52]	Epoch [54/100]	Gen Loss: 1.4939	Disc Loss: 0.8486	Time: 8.21s	
[23:29:00]	Epoch [55/100]	Gen Loss: 1.5014	Disc Loss: 0.8494	Time: 8.26s	

[23:29:08]	Epoch [56/100]	Gen Loss: 1.4885	Disc Loss: 0.8521	Time: 8.23s
[23:29:16]	Epoch [57/100]	Gen Loss: 1.4903	Disc Loss: 0.8495	Time: 8.17s
[23:29:25]	Epoch [58/100]	Gen Loss: 1.4829	Disc Loss: 0.8566	Time: 8.28s
[23:29:33]	Epoch [59/100]	Gen Loss: 1.4831	Disc Loss: 0.8554	Time: 8.21s
[23:29:41]	Epoch [60/100]	Gen Loss: 1.4823	Disc Loss: 0.8615	Time: 8.22s
[23:29:49]	Epoch [61/100]	Gen Loss: 1.4810	Disc Loss: 0.8573	Time: 7.86s
[23:29:57]	Epoch [62/100]	Gen Loss: 1.4750	Disc Loss: 0.8602	Time: 7.58s
[23:30:05]	Epoch [63/100]	Gen Loss: 1.4710	Disc Loss: 0.8675	Time: 7.93s
[23:30:13]	Epoch [64/100]	Gen Loss: 1.4664	Disc Loss: 0.8685	Time: 8.29s
[23:30:21]	Epoch [65/100]	Gen Loss: 1.4586	Disc Loss: 0.8702	Time: 8.23s
[23:30:29]	Epoch [66/100]	Gen Loss: 1.4544	Disc Loss: 0.8748	Time: 8.08s
[23:30:37]	Epoch [67/100]	Gen Loss: 1.4550	Disc Loss: 0.8733	Time: 8.34s
[23:30:46]	Epoch [68/100]	Gen Loss: 1.4493	Disc Loss: 0.8731	Time: 8.12s
[23:30:53]	Epoch [69/100]	Gen Loss: 1.4501	Disc Loss: 0.8745	Time: 7.83s
[23:31:02]	Epoch [70/100]	Gen Loss: 1.4438	Disc Loss: 0.8778	Time: 8.12s
[23:31:10]	Epoch [71/100]	Gen Loss: 1.4432	Disc Loss: 0.8791	Time: 8.09s
[23:31:18]	Epoch [72/100]	Gen Loss: 1.4440	Disc Loss: 0.8807	Time: 7.97s
[23:31:26]	Epoch [73/100]	Gen Loss: 1.4363	Disc Loss: 0.8854	Time: 7.93s
[23:31:34]	Epoch [74/100]	Gen Loss: 1.4403	Disc Loss: 0.8848	Time: 8.20s
[23:31:42]	Epoch [75/100]	Gen Loss: 1.4393	Disc Loss: 0.8846	Time: 8.30s
[23:31:50]	Epoch [76/100]	Gen Loss: 1.4375	Disc Loss: 0.8827	Time: 7.98s
[23:31:58]	Epoch [77/100]	Gen Loss: 1.4352	Disc Loss: 0.8894	Time: 8.21s
[23:32:06]	Epoch [78/100]	Gen Loss: 1.4316	Disc Loss: 0.8899	Time: 8.19s
[23:32:15]	Epoch [79/100]	Gen Loss: 1.4275	Disc Loss: 0.8880	Time: 8.11s
[23:32:23]	Epoch [80/100]	Gen Loss: 1.4262	Disc Loss: 0.8927	Time: 8.10s
[23:32:31]	Epoch [81/100]	Gen Loss: 1.4299	Disc Loss: 0.8923	Time: 8.16s
[23:32:39]	Epoch [82/100]	Gen Loss: 1.4266	Disc Loss: 0.8914	Time: 8.06s
[23:32:47]	Epoch [83/100]	Gen Loss: 1.4269	Disc Loss: 0.8938	Time: 8.07s
[23:32:55]	Epoch [84/100]	Gen Loss: 1.4165	Disc Loss: 0.9003	Time: 8.21s
[23:33:03]	Epoch [85/100]	Gen Loss: 1.4164	Disc Loss: 0.8975	Time: 8.16s
[23:33:11]	Epoch [86/100]	Gen Loss: 1.4186	Disc Loss: 0.8986	Time: 8.06s
[23:33:19]	Epoch [87/100]	Gen Loss: 1.4166	Disc Loss: 0.8992	Time: 8.05s
[23:33:27]	Epoch [88/100]	Gen Loss: 1.4152	Disc Loss: 0.8970	Time: 7.98s
[23:33:35]	Epoch [89/100]	Gen Loss: 1.4118	Disc Loss: 0.9015	Time: 8.02s
[23:33:43]	Epoch [90/100]	Gen Loss: 1.4103	Disc Loss: 0.9040	Time: 7.86s
[23:33:51]	Epoch [91/100]	Gen Loss: 1.4135	Disc Loss: 0.9059	Time: 7.96s
[23:33:59]	Epoch [92/100]	Gen Loss: 1.4082	Disc Loss: 0.9021	Time: 8.04s
[23:34:07]	Epoch [93/100]	Gen Loss: 1.4060	Disc Loss: 0.9060	Time: 8.05s
[23:34:15]	Epoch [94/100]	Gen Loss: 1.4070	Disc Loss: 0.9026	Time: 8.13s
[23:34:23]	Epoch [95/100]	Gen Loss: 1.4100	Disc Loss: 0.9023	Time: 7.92s
[23:34:31]	Epoch [96/100]	Gen Loss: 1.4075	Disc Loss: 0.9029	Time: 7.98s
[23:34:39]	Epoch [97/100]	Gen Loss: 1.4047	Disc Loss: 0.9025	Time: 7.93s
[23:34:47]	Epoch [98/100]	Gen Loss: 1.4054	Disc Loss: 0.9025	Time: 8.06s
[23:34:55]	Epoch [99/100]	Gen Loss: 1.4052	Disc Loss: 0.9040	Time: 8.00s
[23:35:03]	Epoch [100/100]	Gen Loss: 1.4047	Disc Loss: 0.9011	Time: 8.02s

Training complete

Plots saved

Saved final generator snapshot: ./runs/final_epoch_samples.png

[23:35:04] Run Finish

Created Plot_JPGs.zip

[23:35:04] Loss plots exported

Deliverables: Please plot the loss curves and upload at least five handwritten digit images generated by the generator in GAN after training the GAN for 100 epochs. Please make sure to submit your working code files along with the generated images and the plots.

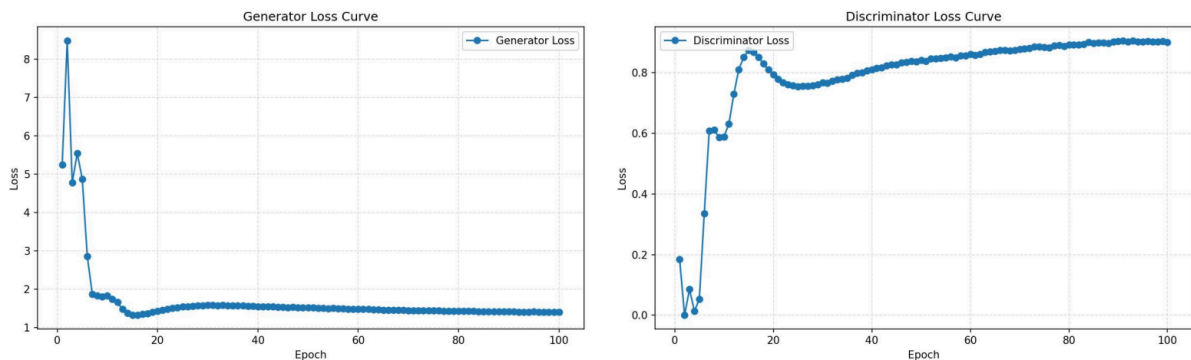
```
In [10]: # Plot Loss Curves
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os

plot_dir = "Plot JPGs"
loss_plot_paths = [
    os.path.join(plot_dir, "Generator_Loss.jpg"),
    os.path.join(plot_dir, "Discriminator_Loss.jpg"),
]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for i, plot_path in enumerate(loss_plot_paths):
    if os.path.exists(plot_path):
        img = mpimg.imread(plot_path)
        axes[i].imshow(img)
        axes[i].axis('off')
    else:
        axes[i].text(0.5, 0.5, 'Plot not found', ha='center', va='center')
        axes[i].axis('off')

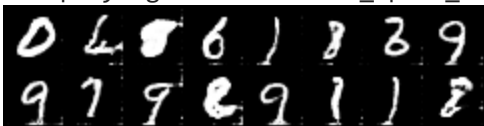
plt.tight_layout()
plt.show()
```



```
In [11]: # Display the last epoch generated image

last_epoch_image_path = './runs/final_epoch_samples.png'
if os.path.exists(last_epoch_image_path):
    print(f"Displaying: {last_epoch_image_path}")
    display(Image(filename=last_epoch_image_path))
else:
    print(f"Last epoch image not found: {last_epoch_image_path}")
```

Displaying: ./runs/final_epoch_samples.png



```
In [12]: # Generate 5 random images to show what the model can do after training

num_images = 5
noise_dim = 100
```

```

export_dir = './runs/export_samples'
os.makedirs(export_dir, exist_ok=True)

generator.eval()
with torch.no_grad():
    sample_noise = torch.randn(num_images, noise_dim, device=device)
    generated_images = generator(sample_noise)

# Analyze generated tensor schema: (batch, channels, height, width)
print(f"Generated tensor shape: {tuple(generated_images.shape)}")
print(f"Value range before denorm: [{generated_images.min().item():.3f}, {generated_images.max().item():.3f}]")

# Convert from tanh range [-1, 1] to display range [0, 1]
generated_images = (generated_images + 1) / 2.0
generated_images = generated_images.clamp(0, 1).cpu()

fig, axes = plt.subplots(1, num_images, figsize=(12, 3))
for i in range(num_images):
    axes[i].imshow(generated_images[i, 0], cmap='gray')
    axes[i].axis('off')
    axes[i].set_title(f"Sample {i+1}")

    sample_path = os.path.join(export_dir, f"generated_sample_{i+1}.png")
    save_image(generated_images[i], sample_path)

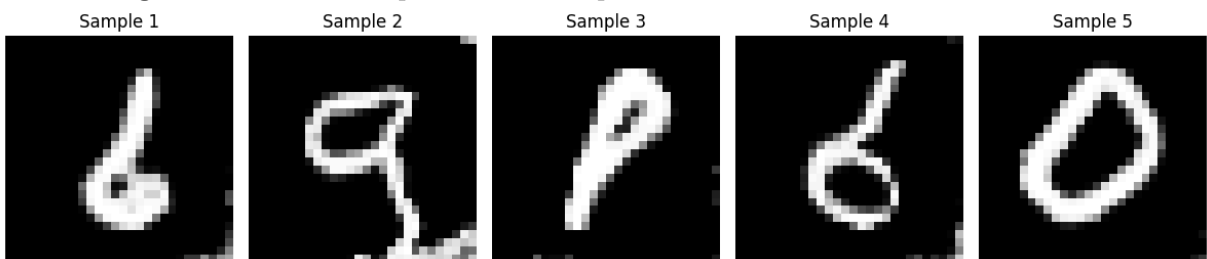
plt.tight_layout()
plt.show()

print(f"Saved {num_images} new samples to: {export_dir}")
print("Run the next cell to export/download the image bundle.")

```

Generated tensor shape: (5, 1, 28, 28)

Value range before denorm: [-1.000, 1.000]



Saved 5 new samples to: ./runs/export_samples

Run the next cell to export/download the image bundle.

```

In [13]: # Export image bundle (last epoch image + 5 newly generated images)

export_dir = './runs/export_samples'
os.makedirs(export_dir, exist_ok=True)

last_epoch_image_path = './runs/final_epoch_samples.png'
if os.path.exists(last_epoch_image_path):
    shutil.copy(last_epoch_image_path, os.path.join(export_dir, 'final_epoch_sample.png'))
    print('Included final_epoch_samples.png in export bundle')
else:
    print('final_epoch_samples.png not found; bundle will include only newly generated images')

```

```
zip_base = './runs/gan_generated_images_bundle'
zip_path = shutil.make_archive(zip_base, 'zip', export_dir)
print(f"Created image bundle: {zip_path}")

if files is not None:
    try:
        files.download(zip_path)
    except Exception as e:
        print(f"Download skipped or failed: {e}")
```

Included final_epoch_samples.png in export bundle

Created image bundle: /content/runs/gan_generated_images_bundle.zip